

SWE40006

Software Deployment and Evolution

Lecture 6
DevOps, CI/CD

Lecture overview (part a)

This lecture will cover,

- Intro to DevOps
 - DevOps Frameworks
 - DevOps tools
 - Infrastructure as Code
-
- Ref: DevOps: Continuous Delivery, Integration, and Deployment with DevOps, Sricharan Vadapalli chap 1, 2

Related Learning objectives

1. Describe the activities in software deployment and apply these in a problem context.
2. Apply knowledge of software environments to plan development and deployment.
3. Describe and classify issues that drive software maintenance.
4. **Describe latest practices in software development deployment and evolution.**
5. **Draw on industry practice to evaluate the maintainability and design consistency of a software system.**

What is DevOps

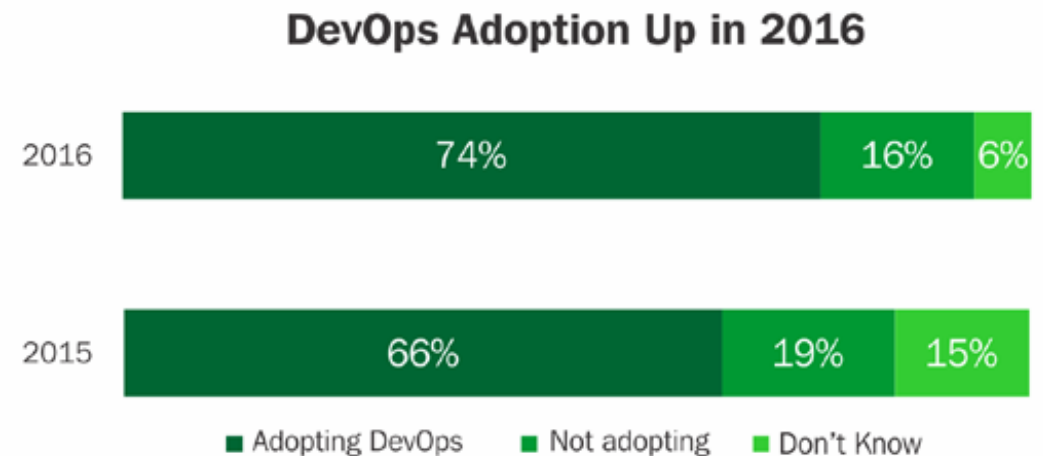


- Might be easier to define what isn't DevOps:
 - Silos of teams/departments/groups/vendors
 - All teams contribute to the SDLC but do not take ownership of / responsibility for the final product.
 - (Job) performance metrics are not related to the success of the software product.
 - Duplicated team admin. Overheads increase overall costs.
 - Multi-skilled workers are boxed into a particular type of task, wasting potential productivity

What is DevOps

- DevOps is Agile, holistic
- Lower risks, consistent environments
- Problems shared
- (Mostly) open-source Dev/test/deploy/monitor tools
- DevOps adoption increasing

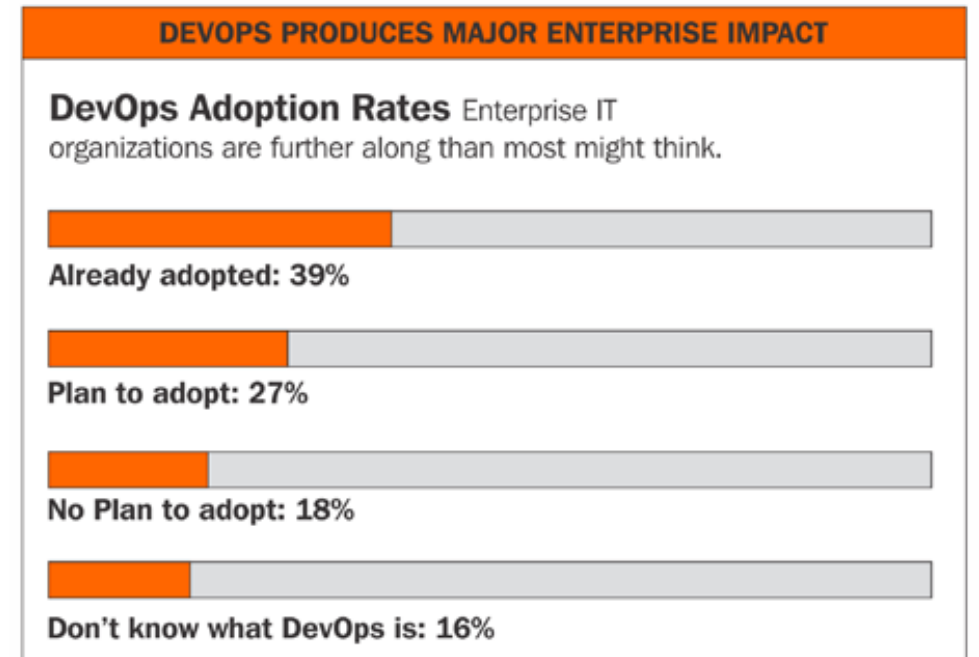
Vadapalli, Sricharan. DevOps: Continuous Delivery, Integration, and Deployment with DevOps, Packt Publishing Ltd, 2018.



Why use DevOps

- Automation of Dev, QA
- Reliable source code/version management (one version).
- Precise config. Management, single tool chain, consistent tool versions.
- Rapid deployment possible
- Portability of code through containerisation

Vadapalli, Sricharan. DevOps: Continuous Delivery, Integration, and Deployment with DevOps, Packt Publishing Ltd, 2018.



Business Benefits

- Ops provides big data, analytics, monetization of data.
- DevOps systems tend to be cloud-based, reducing start-up costs and time to market.
- DevOps suits data science and machine learning – short SDLCs.
- DevOps teams focus on a product instead of a process –
 - Everyone owns the product
- Open-source tools reduce licensing costs
 - Jenkins, Ansible, Chef, Puppet, GitHub, Kubernetes, Docker
- Complete DevOps frameworks provide pre-integrated and configured tools.
 - RedHat Openshift, Azure, AWS

Business Benefits

- Automation of builds, test, deploy
- Stand-alone or integrated processes
- DevOps processes can incorporate SLAs
- App-specific monitoring
- When combined with Cloud, high availability, disaster recovery
- When combined with Cloud, Resilience, scaling
- Ops monitoring collects data on non-functional requirements
- CI/CD reduces downtime (for upgrades) and errors

DevOps tools

Purpose	Tools
Source code management	Git , GitHub, Subversion, and Bitbucket
Build management	Maven , Ant, Make, and MSBuild
Testing tools	JUnit, Selenium, Cucumber and QUnit
Repository management	Nexus, Artifactory, and Docker hub
Continuous integration	Jenkins , Bamboo, TeamCity, and Visual Studio
Configuration provisioning	Chef , Puppet , Ansible , and Salt
Cloud	AWS , Azure , OpenShift, and Rackspace
Deployment management	Rapid Deploy, Code Deploy, and Elastic box
Collaboration	Jira, Team Foundation, and Slack
BI/Monitoring	Kibana, Elasticsearch, and Nagios
Logging	Splunk, Logentries, and Logstash
Release management	Visual Studio, Serena Release, and StackStorm
Container	Linux, Docker , Kubernetes, Swam, AWS, and Azure

DevOps Frameworks

- Coming up:
 - DevOps Processes
 - Infrastructure as Code
- Ref: DevOps: Continuous Delivery, Integration, and Deployment with DevOps, Sricharan Vadapalli Chap 2

DevOps and organisation culture

- DevOps aligns people and processes towards a common goal
- Process frameworks are used at all stages of the SDLC
- Cultural shift required to integrate departments and parts of the SDLC

DevOps processes

- Source code management
- Source code review
- Configuration management
- Build management
- Repository management
- Release management
- Test automation
- Continuous integration
- Continuous delivery
- Continuous deployment

Source code management

- Enables multiple developers to develop code concurrently
- Multiple Dev centres
- Multiple locations
- Versioning of code at a file level
- Parallel development of code branches
- Frequent merging of code into main branch
- Different branches for major and minor releases
- Feature and bug tracking

Source code management allows...

- Coordination of Devs
- Single source of truth for each version/sub-version
- Review of changes before merges
- Author tracking
- Roll-back of versions
- Incremental backup and recovery

Proprietary	Open source
IBM ClearCase	Subversion (SVN)
Perforce	Concurrent Version System (CVS)
PVCS	Git
Team Foundation Server	SCCS
Visual Studio Team Services	Revision control systems
Visual SourceSafe	Bitbucket

Code Review

- Detect memory leaks, buffer overflows, formatting errors
- Types:
 - Formal or informal
 - Pair-programming
 - Tool-assisted (Code walkthroughs)

- Benefits:
 - Improved collaboration
 - Improved quality
 - Improved productivity

Proprietary	Open source
Crucible	Review board
Collaborator	Phabricator
Codacy	Gerrit
Upsource	GitLab
Understand	

Configuration Management

- Enterprise-level configuration
 - Includes Infrastructure Library (ITIL) and Config. Manag. Database (CMDB)
- Identification, verification, maintenance of hardware and software patches and versions
 - Ensures fitness-for-purpose
 - Checks system capabilities and software state
 - Ensures interoperability of systems
- Supports
 - change management processes
 - Automated provisioning on different systems (dev, test, prod)
 - Single version deployment to multiple (different) systems
 - Coordination between Devs and other teams (always works on all machines)

Proprietary	Open source
BMC Software Atrium	Chef
Ivanti (Swinburne)	Puppet
HPE Universal CMDB	Ansible
	Salt
	Juju

Build Management

- Sets up a build environment for integrating software components (as source code) into a working system.
- Manages
 - source code,
 - compilers,
 - dependencies,
 - hardware and
 - other software
- Builds can be:
 - manual,
 - automatic or
 - on-demand (automatic when a script runs indicating a code commit).
- Builds can also be scheduled automatic (CI server) (nightly).

Proprietary	Open source
MSBuild	Ant
Visual Build	Buildr
	Maven
	Gradle
	Grunt
	Make and variants

Build Artefacts Repository Management

- Hosts multiple versions of binaries (builds or executables)
- Advantages:
 - High availability of builds
 - Manages artefact lifecycles
 - Builds shared across many teams
 - Implements retention requirements (for compliance)

Artifacts Repo Man. tools

Sonatype Nexus

Jfrog Artifactory

Apache Archiva

NuGet

Docker Hub

Pulp

NPM

Release Management

- Controls release of build from Dev/Test/Deploy to Ops/Management.
- Release management is driven by feature requests and change requests.
- Orchestrates dev, version control, build, QA, deployment and support.
- Tracks recent deployments in different environments (test/stage/prod)
- Heavily automated

Rel. Man. tools

Electric Cloud

Octopus Deploy

Continuum

Automic

QuikBuild

UrbanCode Release

CA Service Virtualisation, Automation

BMC Release Proc. Manag.

Plutora Release

Serena Release

StackStorm

Rally

Visual Studio

Test Automation

- Run test cases without manual intervention.
- Use an automation tool to schedule tests.
- Reduces the number of manual test cases needed.
- Useful for tedious, repetitive, predictable and known boundary conditions.
- Initial manual test cases (for new code) are used to develop automated tests.
- May test for some security vulnerabilities.

Test automation tools
Visual Studio Test Professional
QTP
SoapUI
TestDrive
FitNesse
Telerik Test Studio
Selenium
TestComplete
Watir
Robotium

Continuous Integration (CI)

- Developers continuously integrate their code in small batches into a common shared code repository
- Code quality is transparent (known to all)
- Bugs are detected quickly and fixed before they accumulate for cause significant problems.
- Requirements:
 - Source code version repository
 - Regular code check-in
 - Automated testing
 - Automated build
 - Build can be deployed at any time

CI tools

Jenkins

TeamCity

Travis

Go CD

Buddy

Bitbucket

Chef

Microsoft TeamCenter

CruiseControl

GitLab CI

Circle CI

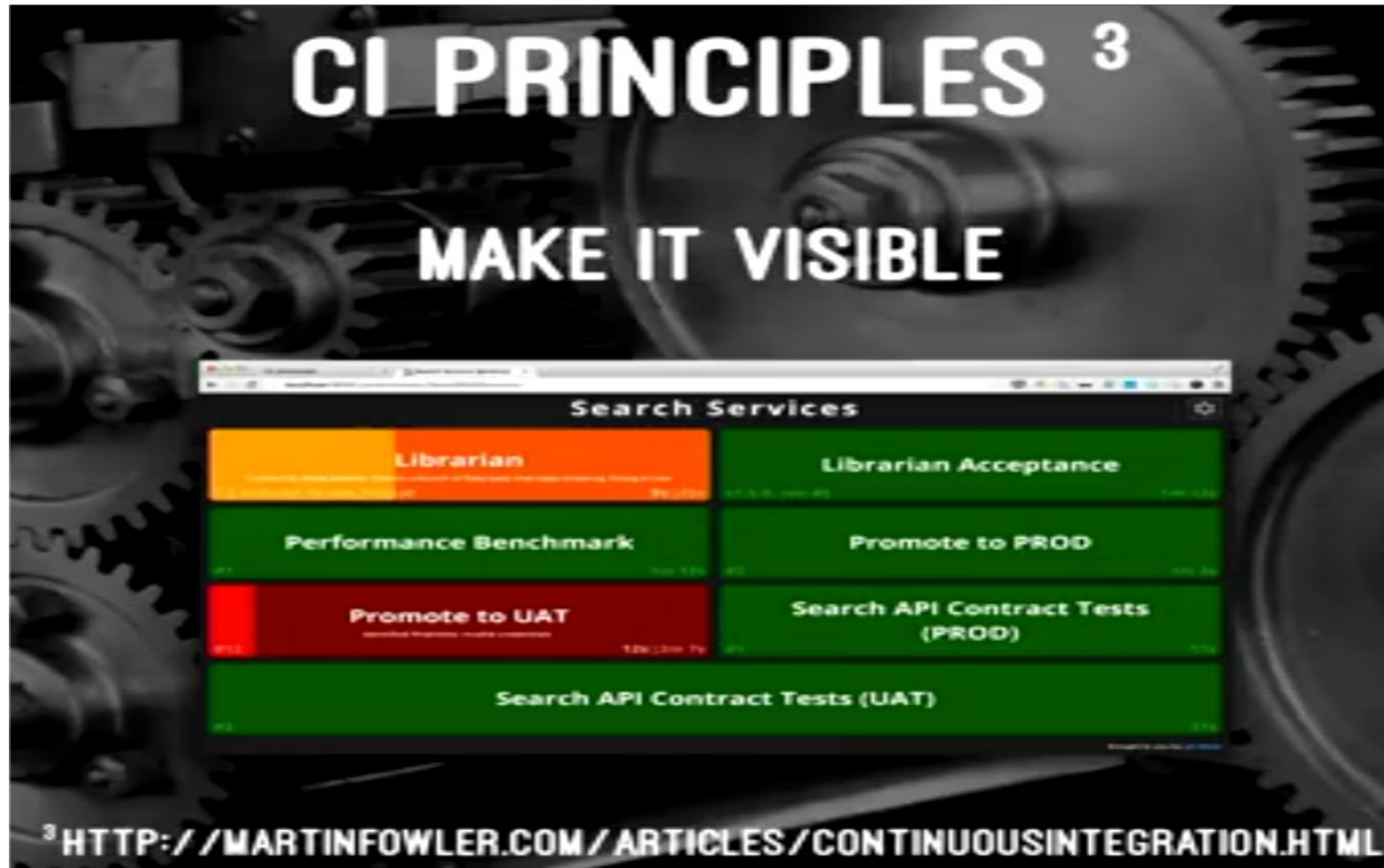
Codship

Bamboo

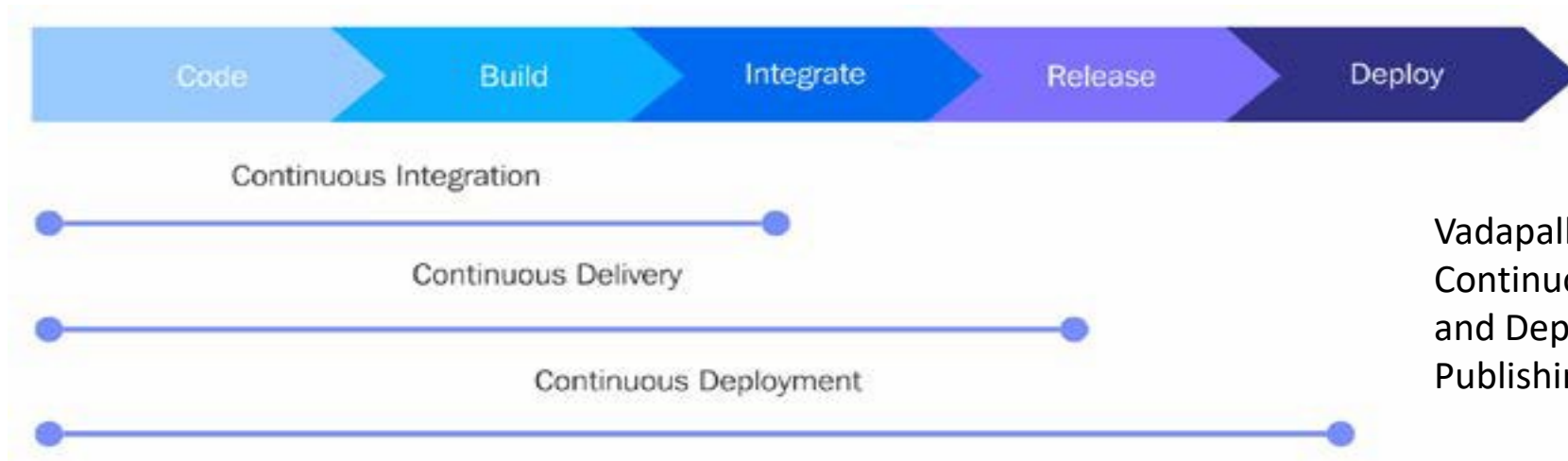
Continuous Integration (CI)



Continuous Integration (CI)



Continuous Delivery (CD includes CI/CD includes CI))



Vadapalli, Sricharan. DevOps: Continuous Delivery, Integration, and Deployment with DevOps, Packt Publishing Ltd, 2018.

- Rapid and reliable development, integration and delivery with minimum effort (very automated).
- Build is deployed to staging environment ready for manual deployment.
- Allows rapid feedback on usability, business logic, functionality.
- Product can be deployed immediately when "suits" choose to.

Continuous Delivery (CD)



Continuous Delivery (CD)



Continuous Delivery (CD)



Continuous Deployment

- Fully-automated SDLC
- CI/CD product deployed to customer/prod environment as soon as it is updated, tested (i.e. passes automated tests)
- Also called "automated application release"
- Benefits:
 - Seriously Agile
 - Every deployment has the latest code updates
 - High quality code (because every code change goes through all the stages of the SDLC), quick troubleshooting.
 - Quick time to market
 - Full integration of Dev/Ops/Test/Prod teams

Infrastructure as Code

- IaC means infrastructure services are defined as code or configuration script.
- IaC automates the
 - setup of cloud services,
 - Installation of software at boot time (first run scripts)
 - Execution of Ansible playbooks, Chef recipes, Puppet manifests
 - Execution of shell scripts, Powershell
 - Start, stop, delete, restart, termination of Cloud-based virtual machines
 - Start, stop, restart services/daemons on an operating system.
 - Download/install/configure services and software (wget, curl, Git pulls, Yum, apt-get)
- IaC scripts should be maintained in code repositories with version control.

Advantages of IaC

- Reduces errors and increases reliability of deployment
- Maintained like code so small changes are tested and can be rolled back if buggy
- Support code auditing and compliance checks
- Support simultaneous deployment to multiple systems
- Support key management (SSH keys, TLS keys) and revocation.

Test automation tools

Ansible Tower

CFEngine

Chef

Puppet

SaltStack

Lecture overview (part b)

This rest of this lecture will cover:

- CI/CD in detail
 - Jenkins
 - Git
 - Maven
- Ref: "Textbook" Chap 3

..... Next lecture: Chap 4

- DevOps Continuous Deployment
 - Chef
 - Ansible
 - Splunk (Ops monitoring)
 - Nagios

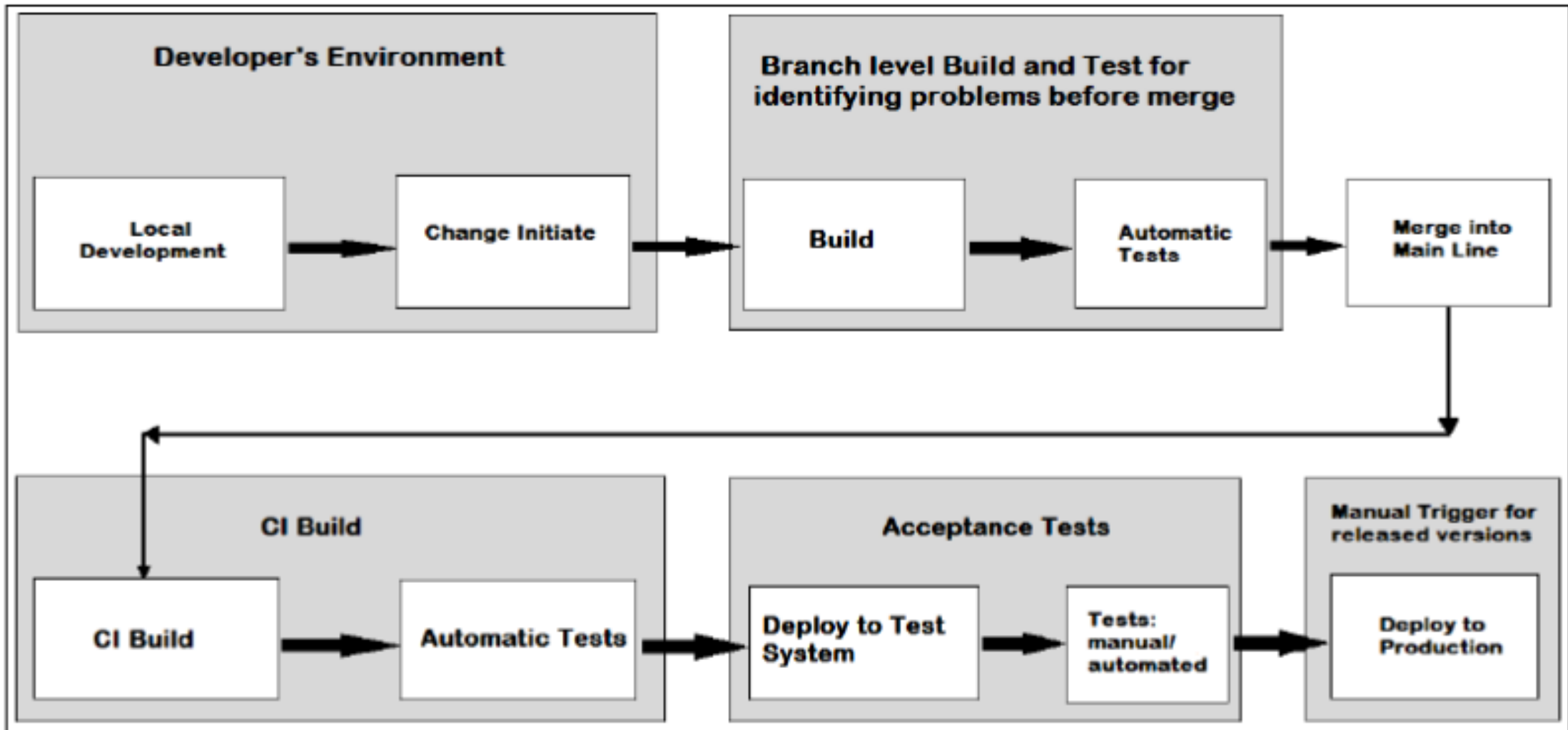
Related Learning objectives

1. Describe the activities in software deployment and apply these in a problem context.
2. Apply knowledge of software environments to plan development and deployment.
3. Describe and classify issues that drive software maintenance.
4. Describe latest practices in software development deployment and evolution.
5. Draw on industry practice to evaluate the maintainability and design consistency of a software system.

Continuous Integration and Delivery (CI/CD)

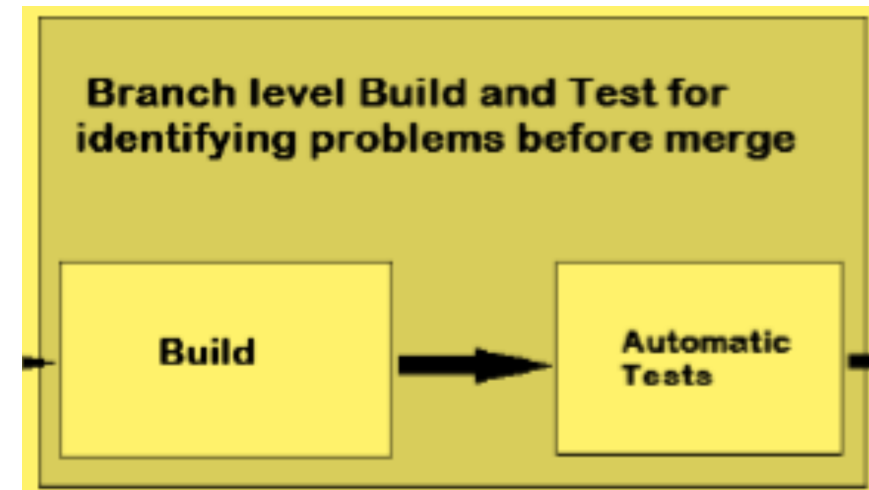
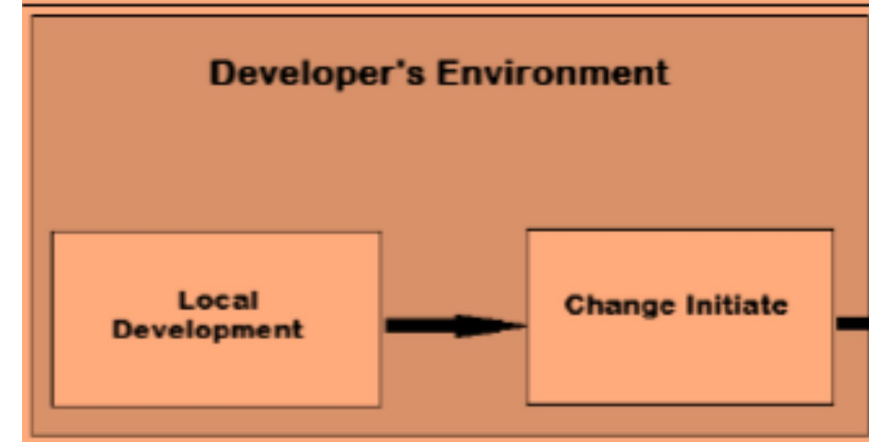
- Multiple developers use Agile methods to:
 - Check all code into a version control system
 - Do code review
 - Re-build code after small changes
 - Use automation to Test/unit-test/integration test code after each change
 - Fix bugs immediately when they are detected
 - Collect metrics of build results and track changes in code repository
- Adding CD:
 - Latest builds which pass testing are ready for deployment
 - Packaging/shipment/deployment can be automated

Another diagram of CI/CD...



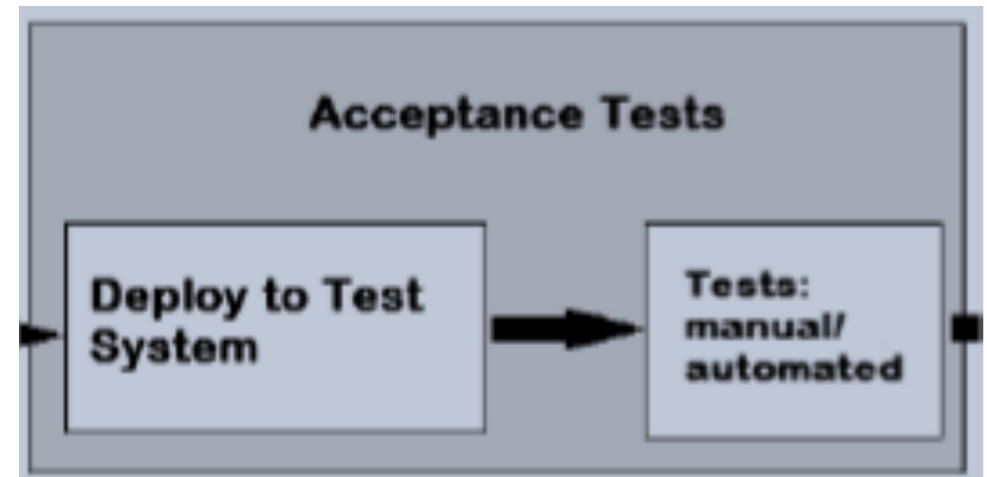
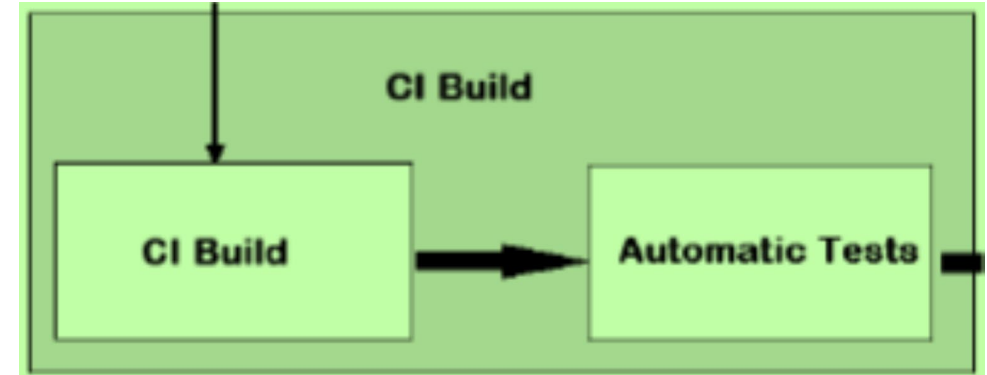
Dev environment

- In local work space (IDE):
 - Change code
 - Unit test
 - Data validation
 - Performance checks
- At the code branch level:
 - Builds,
 - Artefact storage,
 - Integration tests



CI environment

- If all goes well:
 - Merge changes into the main branch
 - Do a full build
 - Full integration tests
 - Quality manager sign-off
- Deploy to test server
 - Performance, compliance and acceptance tests (some automated) start.
 - Delivery manager sign-off
- Deploy (manual)



Some DevOps CI/CD Tools

- Git (code/docs repository)
- Jenkins (CI/CD automation (build, test, deliver, deploy), based on Java)
- Maven (build automation, mainly for Java projects; work with Jenkins)
- Many more ...

Git: <https://git-scm.com/>

Jenkins: <https://www.jenkins.io/>

Maven: <https://maven.apache.org/>

Jenkins is still widely used, but there are new alternatives ...

Jenkins' new alternatives

Here are some of the best CI/CD tools to consider (cloud orientation):

- [Northflank](#) → A modern alternative with cloud-first CI/CD and Kubernetes support.
- [GitHub Actions](#) → Built into GitHub, great for repository-based workflows.
- [GitLab CI/CD](#) → Deep integration with GitLab and flexible automation.
- [CircleCI](#) → Known for speed and cloud-based builds.
- [Harness](#) → AI-powered CI/CD with cost and security optimization.
- [Travis CI](#) → Simple setup, popular for open-source projects.
- [Bitbucket Pipelines](#) → Direct integration with BitBucket repositories.
- [TeamCity](#) → Self-hosted option with customizable build processes.
- [AWS CodePipeline](#) → Works well with AWS services.
- [Azure DevOps](#) → Microsoft's CI/CD solution with broad enterprise support.

Details: **Jenkins alternatives in 2025** (<https://northflank.com/blog/jenkins-alternatives-2025#:~:text=Don't%20get%20me%20wrong,fit%20modern%20workflows%20makes%20sense>).

Suggestions for the mid-semester break

1. Try to complete deployment task 4
2. Start the group project
3. Start/continue the personal review of unit content (for the week 9 test): lectures, tutorials, readings, etc

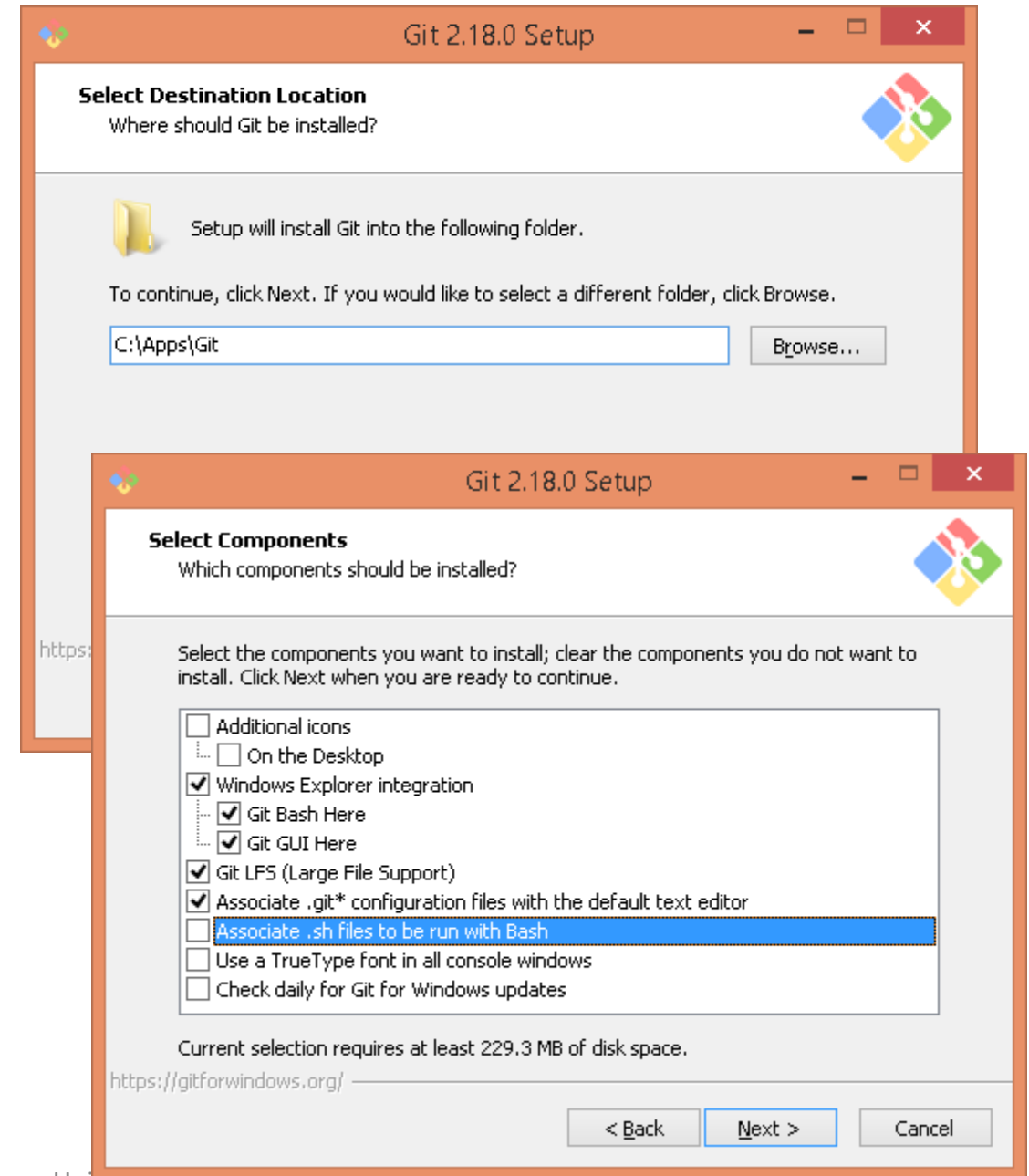
James' tool installation exercise from a while back

The remaining slides document James Hamlyn-Harris' exercise/experience for installing some DevOps CI/CD tools from a while back. Some info is outdated, but may still have some reference value:

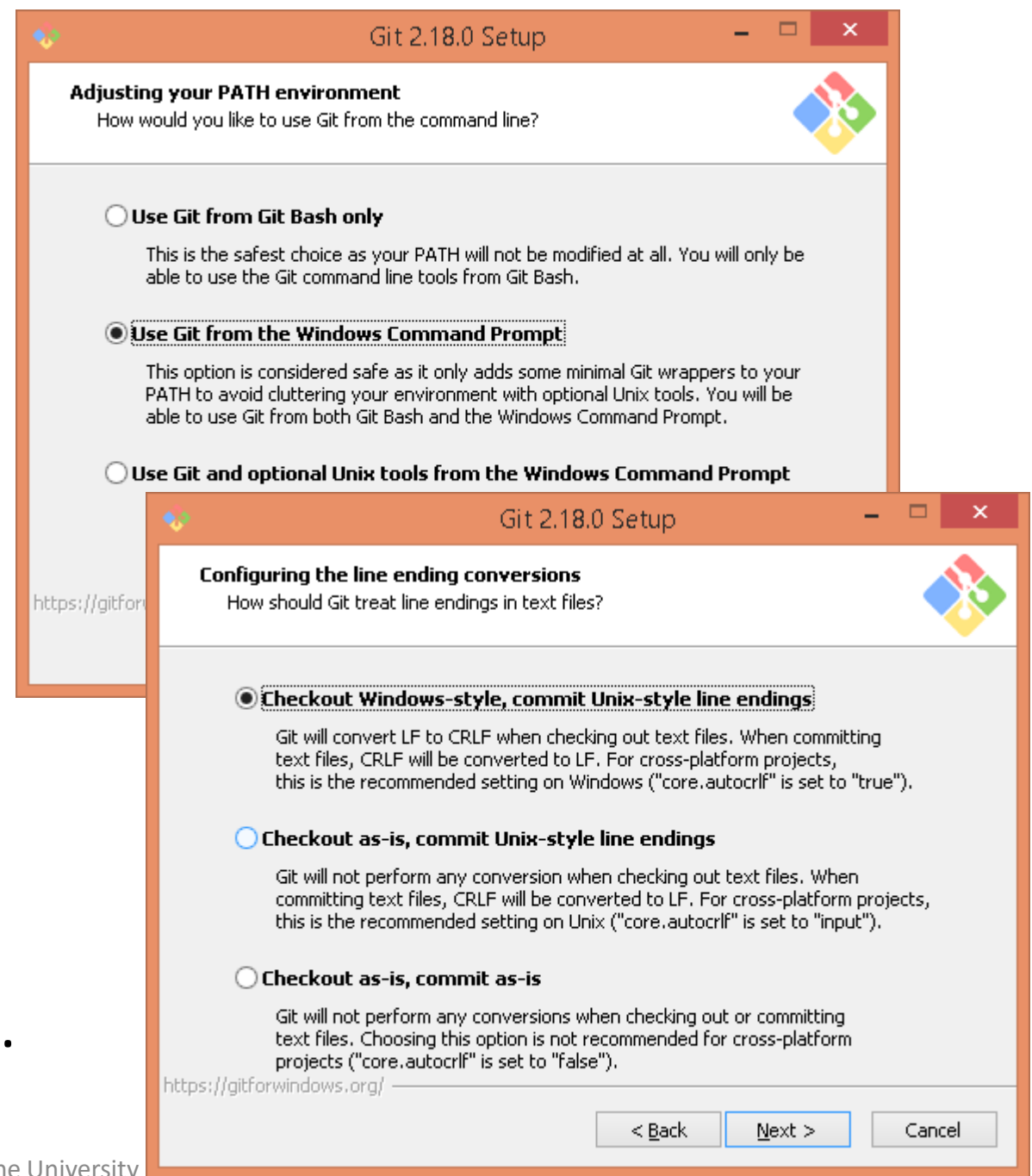
- Git
- Jenkins
- Maven

Git binaries

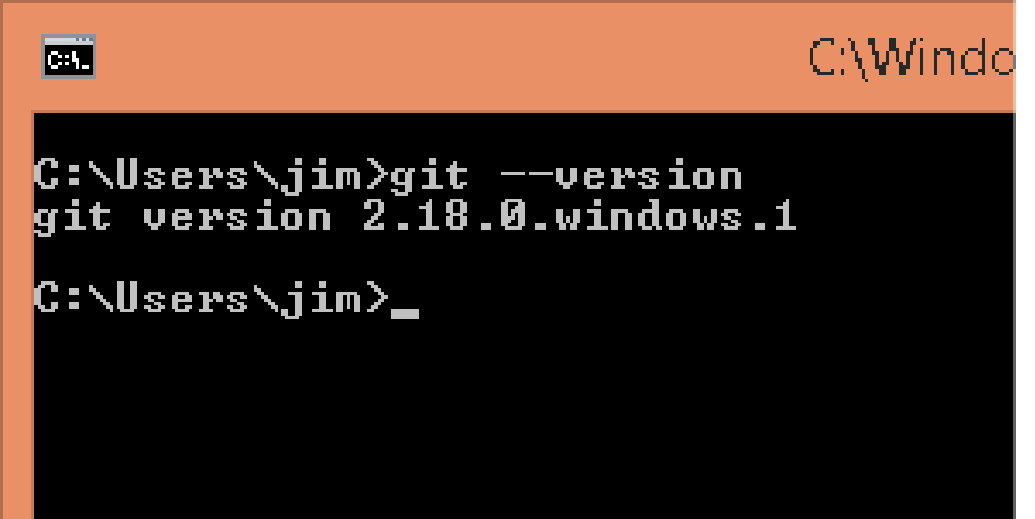
- <https://www.git-scm.com/>
- Download (on Canvas)
- Install to a path with NO spaces.
e.g. C:\Apps\Git
- Uncheck the .sh file association



- Choose the Windows command prompt option – if you're using Windows.
- Leave the defaults for line termination characters
- Other defaults can remain in place.



- To test, open a Windows command prompt/terminal window
- <Win>+R cmd <Enter>
- Type in **git --version** to see if it's installed.



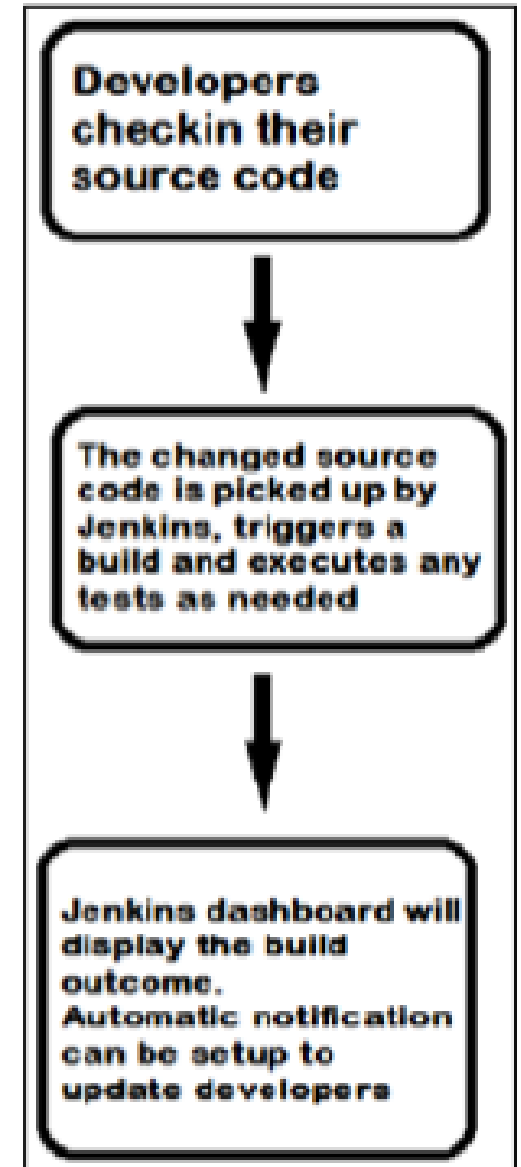
```
C:\Users\jim>git --version
git version 2.18.0.windows.1
C:\Users\jim>_
```

CI server: Jenkins

- Jenkins is an automation server. CI is one use for it.
- Requirements:
 - Java run-time or Java Development Kit (JDK) **version 8** (not 10!)
- Can be installed on a box (stand-alone), on a Java-based web server (servlets) or on a server which provides services in a client-server architecture.
- To run as a stand-alone process:

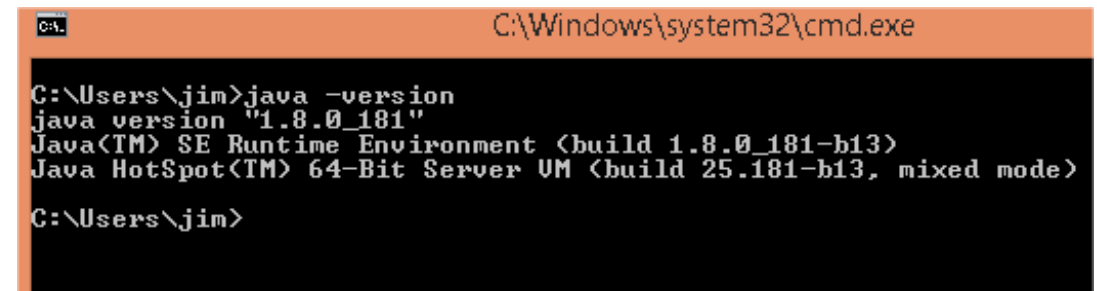
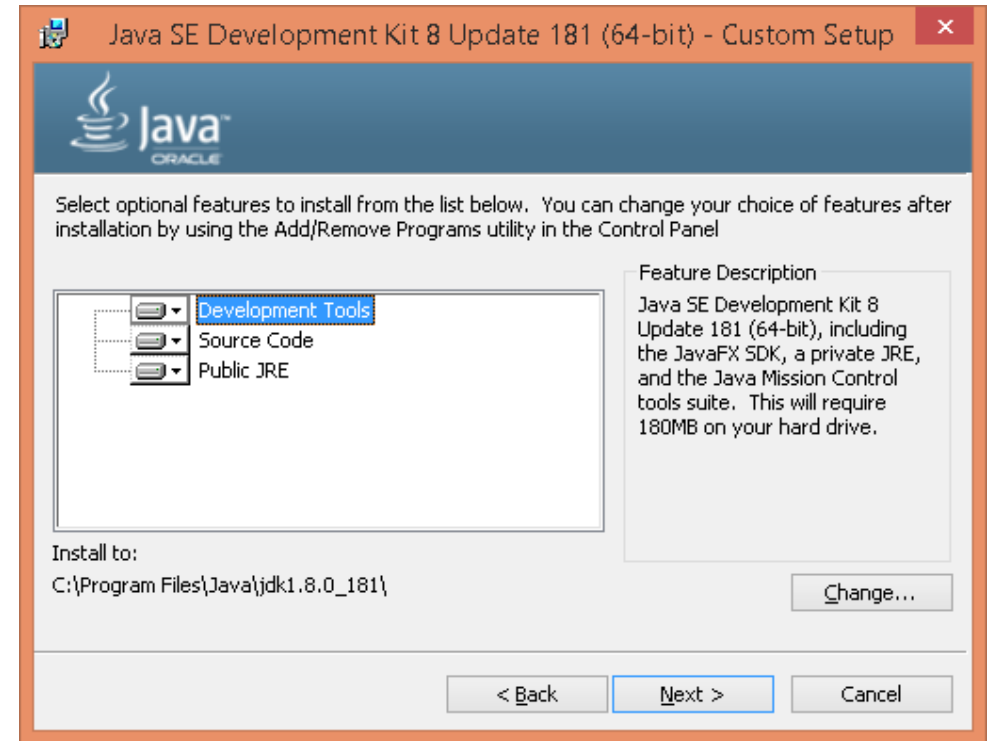
```
java -jar "<jenkins path>jenkins.war"
```

- The textbook – chapter 3 has screen-shots of the process of using Jenkins for a simple deployment.



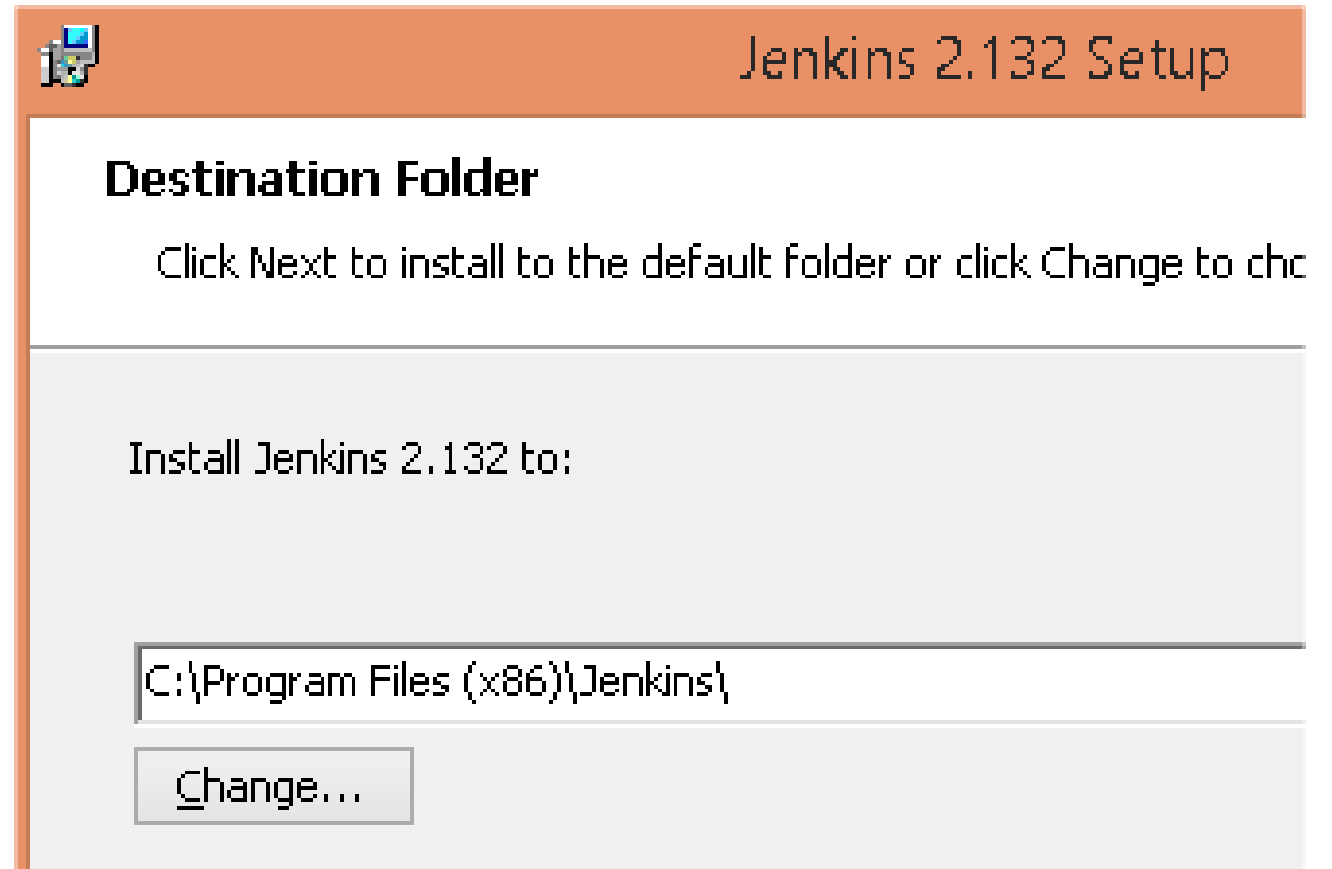
If you haven't got Java...

- Install JDK on your laptop. Best if you remove all previous versions of Java first.
- You can accept the defaults
- You can skip the Next Steps button – just launches a web page with the documentation.
- Takes about 5 mins.



Install Jenkins

- Download Jenkins (copy on Canvas)
 - I got the .msi in a zip file
 - Mac users-get the .pkg file
- Open a terminal/CUI/DOS window
 - The default path is OK
 - Jenkins may launch a browser to load localhost:8080. If this does not load, you will have to launch jenkins manually.



Starting Jenkins...(for Windows)

- Open a terminal/DOS/CUI console:

```
<Win>+R cmd<enter>
```

```
java -version
```

- assuming this is your Jenkins dir:

```
C:\Program Files (x86)\Jenkins
```

```
java -jar "C:\Program Files (x86)\Jenkins\jenkins.war"
```

- If you get an exception, it's not necessarily a problem.
- Open a web browser and go to localhost:8080

If you get DOS errors,
check that the
characters: - " are the
ASCII ones, not some
MSOffice
substitutions

Installed and started without intervention on Jim's Win7 work desktop.

Installed and didn't start on Jim's Win8.1 clean install desktop – go figure. Had to do a manual launch (next slide) which raised exception (even though the Tomcat server started and everything except Maven worked correctly.

Bottom line – if something unexpected happens, try installing on a different computer.

```
C:\Windows\system32\cmd.exe - java -jar "C:\Program Files (x86)\Jenkins\jenki...

Running from: C:\Program Files (x86)\Jenkins\jenkins.war
webroot: $user.home/.jenkins
Jul 22, 2018 5:42:16 PM org.eclipse.jetty.util.log.Log initialized
INFO: Logging initialized @2417ms to org.eclipse.jetty.util.log.JavaUtilLog
Jul 22, 2018 5:42:17 PM winstone.Logger logInternal
INFO: Beginning extraction from war file
+133mJul 22, 2018 5:42:17 PM org.eclipse.jetty.server.handler.ContextHandler set
ContextPath
WARNING: Empty contextPath
+10mJul 22, 2018 5:42:17 PM org.eclipse.jetty.server.Server doStart
INFO: jetty-9.4.z-SNAPSHOT; built: 2018-06-05T18:24:03.829Z; git: d5fc0523cfa96b
febfbd19606cad384d772f04c; jvm 1.8.0_181-b13
Jul 22, 2018 5:42:27 PM org.eclipse.jetty.webapp.StandardDescriptorProcessor vis
itServlet
INFO: NO JSP Support for /, did not find org.eclipse.jetty.jsp.JettyJspServlet
Jul 22, 2018 5:42:27 PM org.eclipse.jetty.server.session.DefaultSessionIdManager
doStart
INFO: DefaultSessionIdManager workerName=node0
Jul 22, 2018 5:42:27 PM org.eclipse.jetty.server.session.DefaultSessionIdManager
doStart
INFO: No SessionScavenger set, using defaults
Jul 22, 2018 5:42:27 PM org.eclipse.jetty.server.ses
ging
INFO: node0 Scavenging every 600000ms
```

After a manual launch of Jenkins
this raised exceptions. Very strange.
On a different computer/OS we
never even saw this window

Go to the
highlighted file
and get the
password.
Paste it here:

You can copy the file path into Win
explorer (<Win> + E), and then open
the file with notepad.

Getting Started

Unlock Jenkins

First-run activation

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
C:\Program Files (x86)\Jenkins\secrets
```

```
\initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

First-run activation

- Allow Jenkins to install the default plug-ins

Install suggested plugins

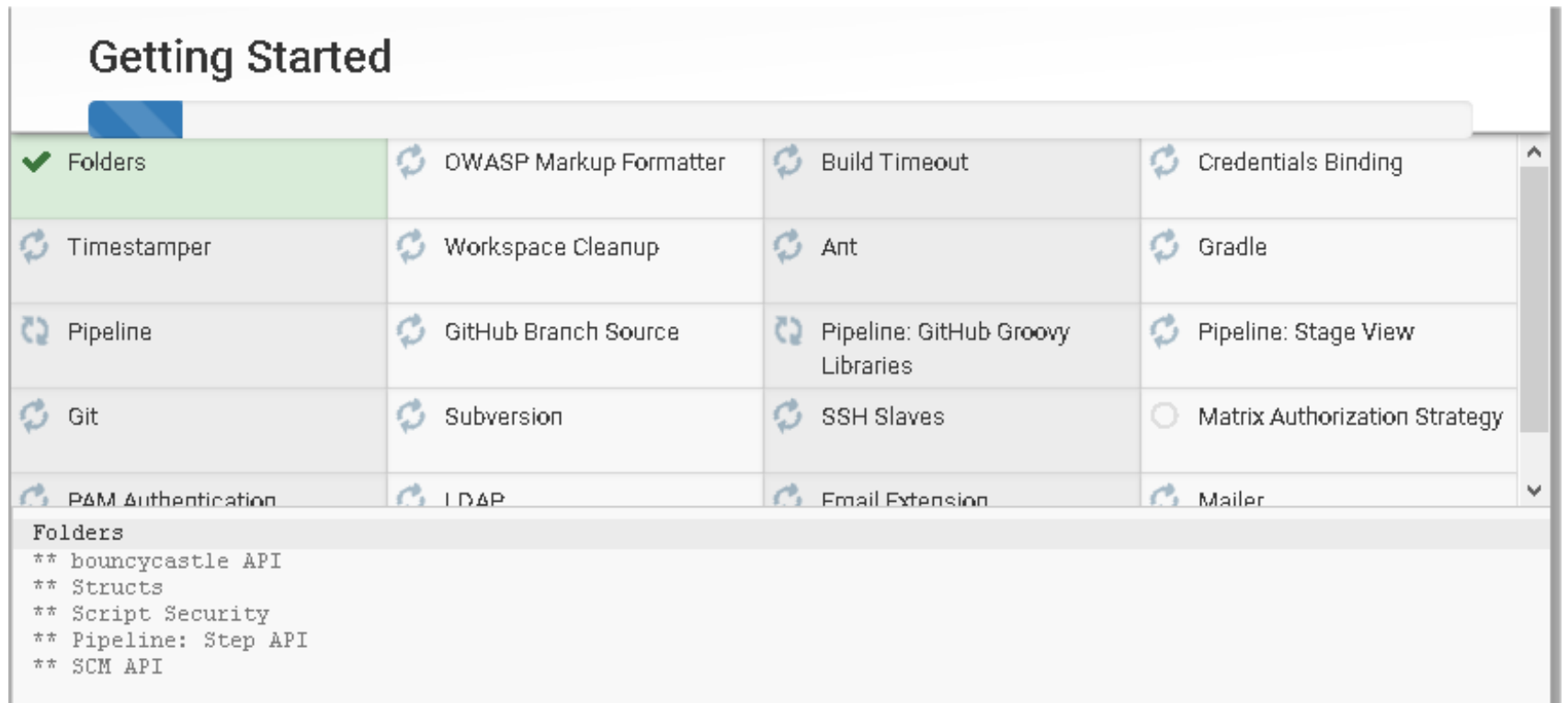
Install plugins the Jenkins community finds most useful.

More first-run

- Jenkins will slowly install the plug-ins, showing a progress bar and twirly arrows.

- May takes a while ...
- About 5 mins on a Swin network.

Getting Started



✓ Folders	OWASP Markup Formatter	Build Timeout	Credentials Binding
Timestampers	Workspace Cleanup	Ant	Gradle
Pipeline	GitHub Branch Source	Pipeline: GitHub Groovy Libraries	Pipeline: Stage View
Git	Subversion	SSH Slaves	Matrix Authorization Strategy
PAM Authentication	LDAP	Email Extension	Mailer

Folders

```

** bouncycastle API
** Structs
** Script Security
** Pipeline: Step API
** SCM API

```

Getting Started

Create First Admin User

Username:

Password:

Confirm password:

Full name:

E-mail address:

Use these credentials
on subsequent runs

Jenkins 2.131

[Continue as admin](#)

[Save and Continue](#)

Pick a user name and password you will remember.
A real e-mail address will help you get back in if you forget.

Jenkins Instance Configuration

- Jenkins URL: `http://localhost:8080`
- And you're done!
- If you don't see this: 

you need to turn down your security settings – whitelist *localhost:8080* for pop-ups and cookies.



Welcome to Jenkins!

Sign in

☐ Keep me signed in

Create new job

- Create a new job
- Select a freestyle project [OK]
- Under Source Code Management, select Git. But wait – need to start Git on the computer
- The installer should have created a shortcut called Git Bash – run it

Welcome to Jenkins!

Please **create new jobs** to get started.

Enter an item name

» Required field



Freestyle project

This is the central feature of Jenkins. Jenkins is something other than software build.

Git



Git Bash **NEW**



Git CMD **NEW**



Git GUI

Git Bash

- Try **git --version**

- **mkdir GitDemo**

- **cd GitDemo**

- **git init**
(creates a repo.)

```
jim@Cooler MINGW64 ~
$ git --version
git version 2.18.0.windows.1
```

```
jim@Cooler MINGW64 ~
$
```

```
jim@Cooler MINGW64 ~
$ mkdir GitDemo
```

```
jim@Cooler MINGW64 ~
$ cd GitDemo
```

```
jim@Cooler MINGW64 ~/GitDemo
$
```

```
$ git init
Initialized empty Git repository in C:/Users/jim/GitDemo/.git/

jim@Cooler MINGW64 ~/GitDemo (master)
$
```

Thought you could
avoid Linux? No way!

Setting up Git user account

- in Git Bash:

```
git config --global user.name "Jim Hamlyn-Harris"
```

- to verify it worked:

```
git config --global user.name
```

- To add your e-mail address:

```
git config --global user.email "jhamlynharris@swin.edu.au"
```

- and `git config --global user.email` to see if it worked.

```
jim@Cooler MINGW64 ~/GitDemo (master)
$ git config --global user.name "James Hamlyn-Harris"

jim@Cooler MINGW64 ~/GitDemo (master)
$ git config --global user.name
James Hamlyn-Harris

jim@Cooler MINGW64 ~/GitDemo (master)
$ git config --global user.email "jhamlynharris@swin.edu.au"

jim@Cooler MINGW64 ~/GitDemo (master)
$ git config --global user.email
jhamlynharris@swin.edu.au

jim@Cooler MINGW64 ~/GitDemo (master)
$ |
```

First commit

- Create a file

```
echo "This is a README file" >> README.md
```

- Add it to the git staging area

```
git add README.md
```

```
jim@Cooler MINGW64 ~/GitDemo (master)
$ echo "This is a README file" >> README.md

jim@Cooler MINGW64 ~/GitDemo (master)
$ git add README.md
warning: LF will be replaced by CRLF in README.md.
The file will have its original line endings in your working directory.
```

- Move the file from the staging area to the repo. main branch:

```
git commit -m "README version 1"
```

```
$ git commit -m "README version 1"
[master (root-commit) 7b9efa0] README version 1
1 file changed, 1 insertion(+)
create mode 100644 README.md

jim@Cooler MINGW64 ~/GitDemo (master)
$
```

Back to Jenkins

- In the Source Code Management part of the config page, add the local path (file system path) to your Repository URL:
- If you want to connect to GitHub, put the URL here instead.

Source Code Management

☐ None
☒ Git

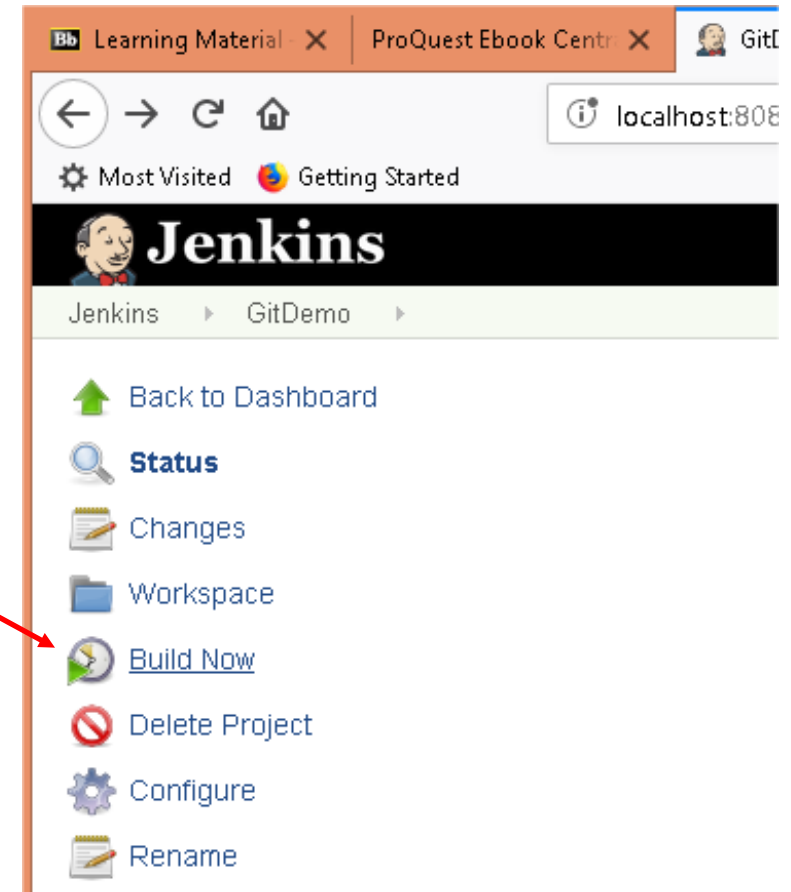
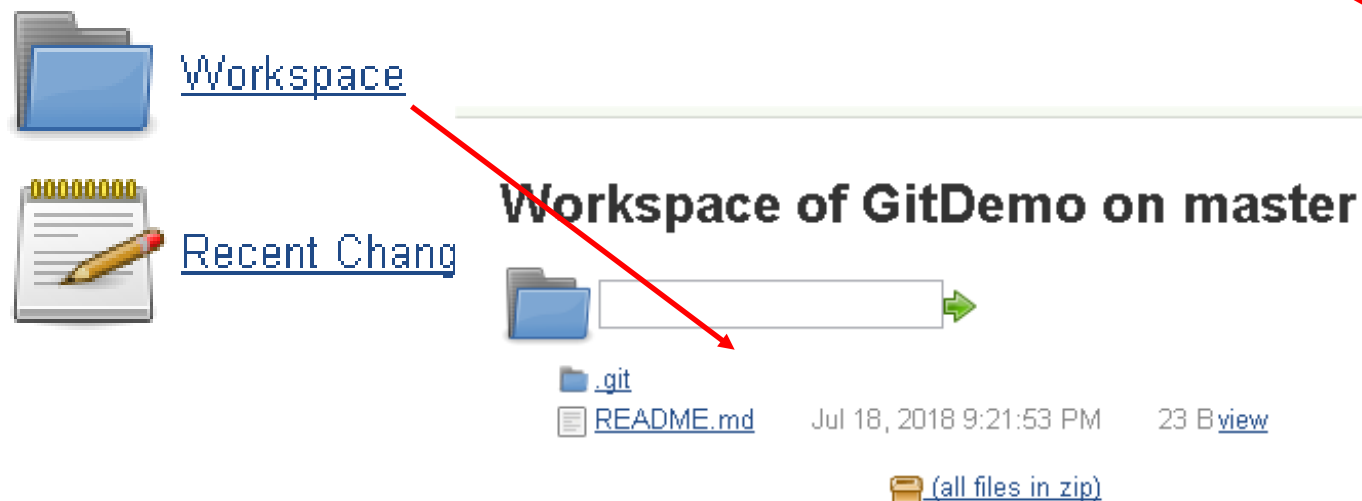
Repositories

Repository URL

Credentials

Run your first Build

- in `http://localhost:8080/job/GitDemo/`
- Build the first version:
- You can now see the readme file in the Workspace:



Try changing the file (from Git Bash)

```
jim@Cooler MINGW64 ~/GitDemo (master)
$ echo "adding an extra line to the readme file" >> README.md

jim@Cooler MINGW64 ~/GitDemo (master)
```

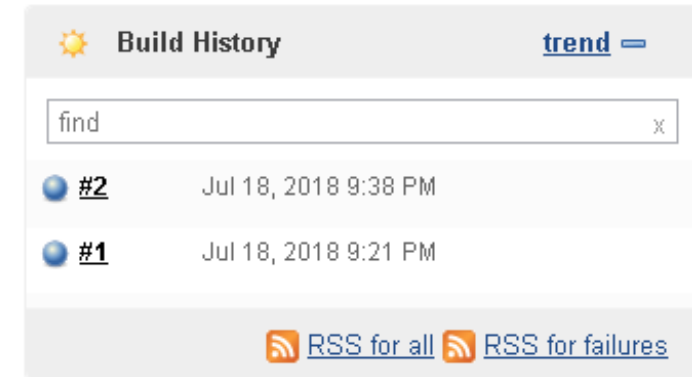
- Back in Jenkins, if you [view](#) the readme file, there is no change.
- Back in Git Bash, add and commit the new version:

```
$ git add README.md
warning: LF will be replaced by CRLF in README.md.
The file will have its original line endings in your working directory.

jim@Cooler MINGW64 ~/GitDemo (master)
$ git commit -m "README version 2"
[master 2804ba6] README version 2
1 file changed, 1 insertion(+)
```

Build 2

- Back in Jenkins, run the build script again.
- view now shows that the repo has the latest version.
- The next step is to add automated compiling with Maven.

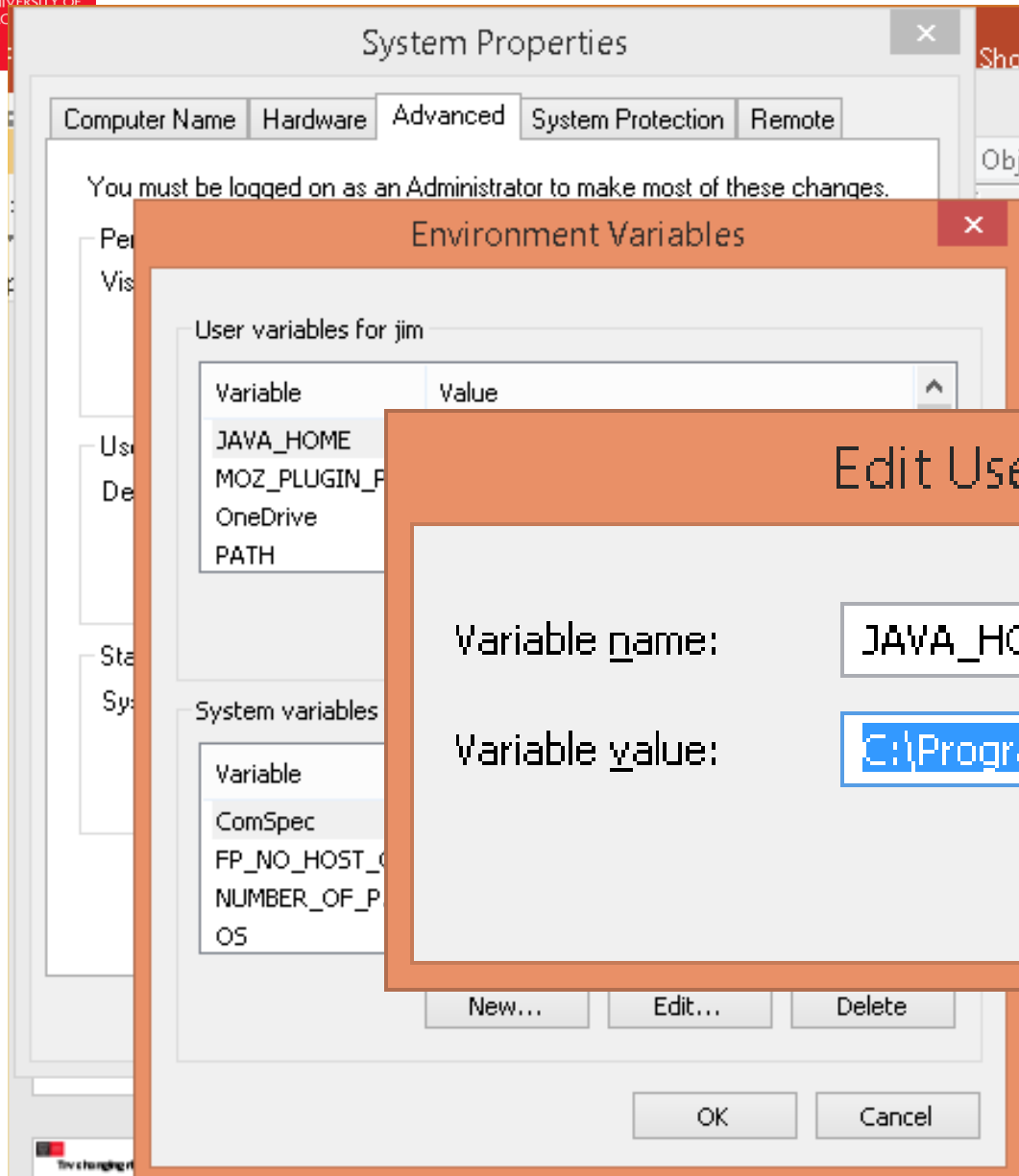


Maven

- (from Wikipedia) “**Maven** is a build automation tool used primarily for Java projects. **Maven** addresses two aspects of building software: first, it describes how software **is** built, and second, it describes its dependencies. ... **Maven can** also be used to build and manage projects written in C#, Ruby, Scala, and other languages.”
- Maven uses XML documents and ANT scriptlets to manage dependencies, builds and projects.
- It currently supports projects using Java, .NET (C#) and C/C++

Install Maven (windows)

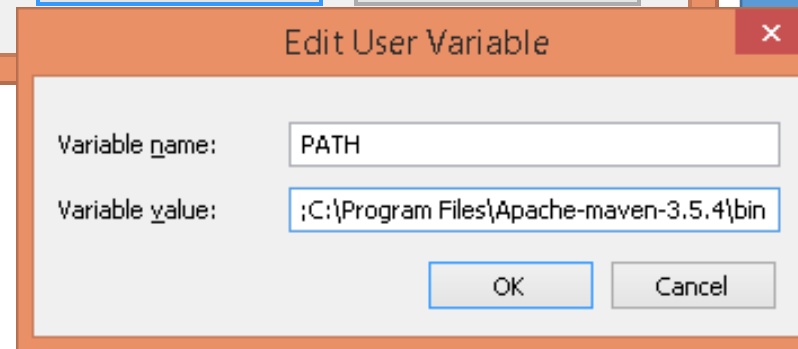
- Install Maven
- Get the .zip file from Canvas
- Maven is all written in Java – extract the zip contents to **c:\Program Files** (or \usr\sbin\ or whatever)
 - Or try C:\Apps\apache-maven-3.5.4
- In the **...Apache-maven-3.5.4\bin** there is a file called **mvn.cmd**
- If you set the paths right this will become executable.



Add the path (C:\Program Files\Apache-maven-3.5.4\)
Check the PDFs on Blackboard for more help. Java paths are fiddly but you need to learn how to do them. Stack Overflow is good for error messages.

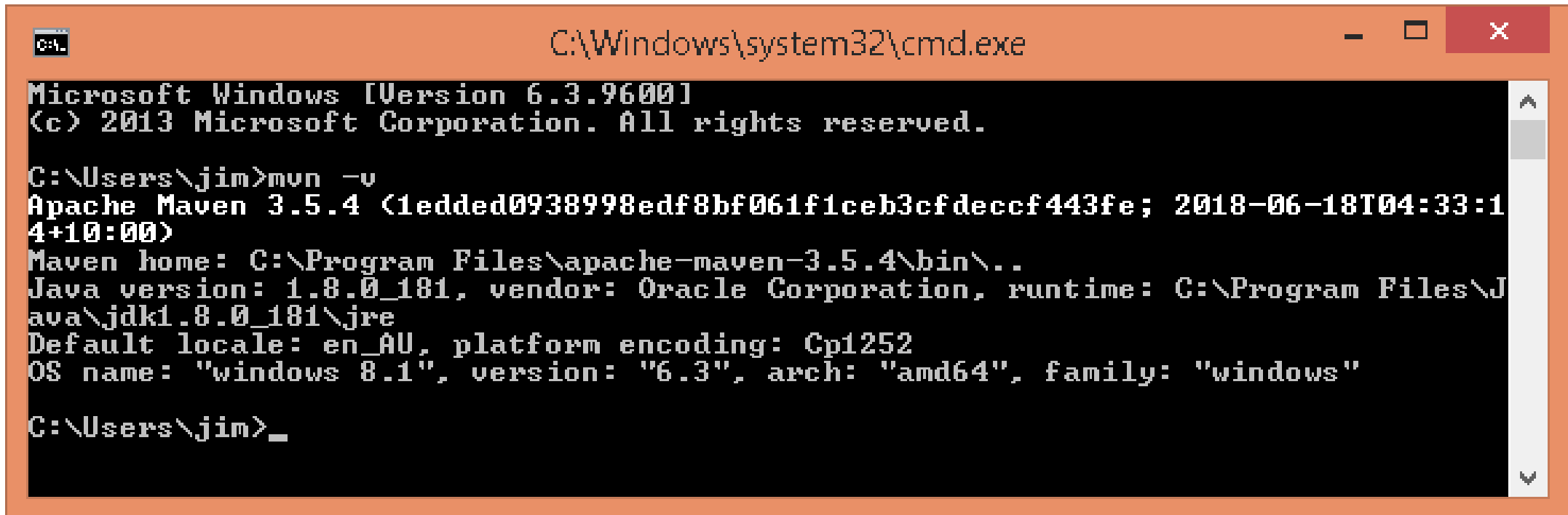
no bin\

Also make sure the PATH includes the location of the Maven binaries:



Test Maven

- Open a new CMD/Terminal window
- **mvn -v**



```

C:\Windows\system32\cmd.exe

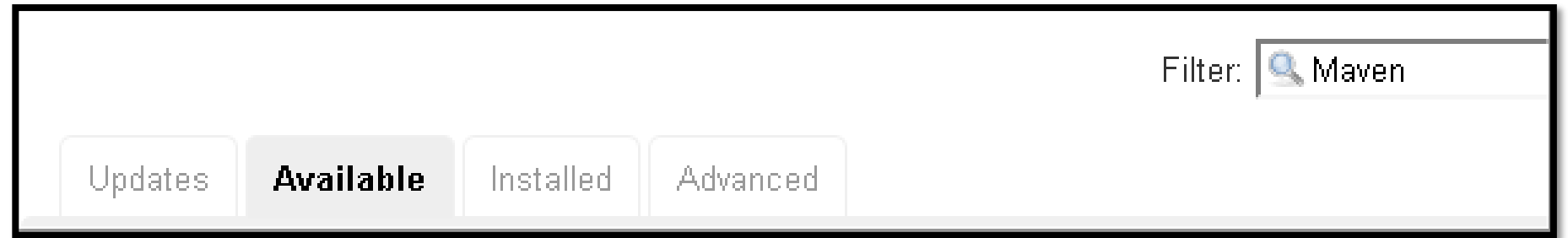
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\jim>mvn -v
Apache Maven 3.5.4 (1edded0938998edf8bf061f1ceb3cfdeccf443fe; 2018-06-18T04:33:14+10:00)
Maven home: C:\Program Files\apache-maven-3.5.4\bin\..
Java version: 1.8.0_181, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk1.8.0_181\jre
Default locale: en_AU, platform encoding: Cp1252
OS name: "windows 8.1", version: "6.3", arch: "amd64", family: "windows"

C:\Users\jim>_
  
```

Back to Jenkins...

- Log back in if necessary
- Click on **Manage Jenkins**
- Select **Manage Plugins**
- Click on the **Available** tab and Filter for Maven



- Select:
 - Maven Deployment Linker,
 - Pipeline Maven Integration,
 - Maven Repository Server
 - and Click **Install without restart**

Scroll down the page to find these.

Watch the grass grow...

- Not updating?
- Try clicking on the restart option.
- ENABLE AUTO REFRESH is good too

Installing Plugins/Upgrades

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

JDK Tool	 Success
Command Agent Launcher	 Success
Config File Provider	 Success
Pipeline Maven Integration	 Success
Windows Slaves	 Success
Javadoc	 Success
Async Http Client	 Success
External Monitor Job Type	 Success
Maven Deployment Linker	 Success
Maven Integration	 Pending
Maven Repository Server	 Pending

- Go back to Manage Jenkins,
- Global Tool Configuration
 - scroll down to the bottom of the page
- Click on
 - Add Maven
 - Un-check Install automatically

Ant

Ant installations

Add Ant

List of Ant installations on this system

Maven

Maven installations

Add Maven

List of Maven installations on this system

Docker

Docker installations

Add Docker

List of Docker installations on this system

Save

Apply

Set the Maven Home path (installation directory)

- Add a Maven **name** (Maven-local)
- Add a MAVEN_HOME (C:\Program Files\Apache-maven-3.5.4\)
 - or where maven is
 - Apply, Save
- This will install an instance of Maven for each project you create.

Maven

Maven installations

Add Maven

Maven

Name

MAVEN_HOME

☐ Install automatically

Delete Maven

Save **Apply**

Testing (*itcuties* example)

- Check that this project is still present on GitHub:
- <https://github.com/itcuties/Axis-2-client.git>

Go there in a web browser

- You will copy the link into Jenkins:
 - In Jenkins, click **New Item**
 - Select **Freestyle project**
 - Enter an item **name** (Test-Build) [OK]

Enter an item name

Test-Build

» Required field



Freestyle project

This is the central feature of Jenkins. Jenkins is used for something other than software bu



Maven project

Build a maven project. Jenkins takes advan

Source code management

- Select **Git**
- paste into the **Repository URL**
- <https://github.com/itcuties/Axis-2-client.git>

Jenkins will try to connect to the repo.

- Have a look at the **Additional Behaviours** Add button.
 - We won't use any of these – in previous versions of Jenkins these were on the main config page.
- Scroll down to **Build Triggers** – note the options for triggering an automated build.
 - we won't use these as we do not control the *itcuties* Git site. Otherwise every time an itcuties dev updated the web repo we could end up automatically running build

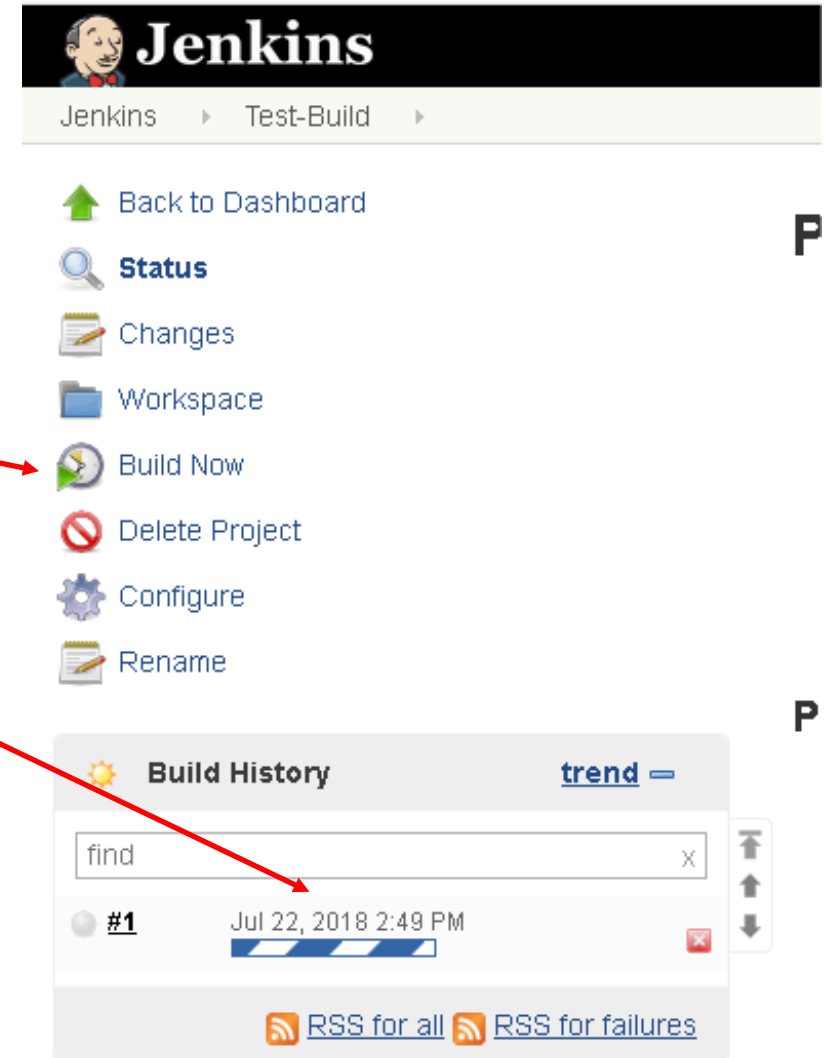
Problems? Try re-starting Git – generally caused if Git path has a space in it or if environment variables aren't quite right. In Jenkins, get to Global Tool Configuration and add the "Path to Git executable" to C:\Apps\Git\bin\git.exe

- In the **Build** section,
- click on **Add Build Step**, and select
 - **Invoke top-level Maven targets**
- Add the word: *compile* to the **Goals** field
- click on the **Advanced** button in the **Build** section
- Back in GitHub, find the pom.xml file (in Axis-2-client/HighScoresServiceClient)
- in Jenkins, Add the path: *HighScoresServiceClient\pom.xml* to the POM field.
- Scroll down to the bottom of the page, **Apply, Save**

Doesn't work – mvn command line causes error.
Builds ok if this step is omitted.

Back in the Project...

- Click Build Now
- You can see the console output of the build process by right-clicking on the progress bar.
 - Jenkins pulls the project from GitHub
 - Assigns a version number
 - Copies resources
 - ~~The Maven plugin Compiles the source files~~



Errors...

- *[ERROR] The goal you specified requires a project to execute but there is no POM in this directory (C:\Program Files (x86)\Jenkins\workspace\Test-Build). Please verify you invoked Maven from the correct directory. -> [Help 1]*
- To fix this error: Check that the POM field in the Build section of the Project Config is correct: HighScoresServiceClient\
- Another error: *\Jenkins\workspace\Test-Build\HighScoresServiceClient was unexpected at this time*
- Check the JAVA_HOME environment variable (using a DOS prompt). If it's missing, log in as admin and set it for the system to: C:\Program Files\Java\jdk1.8.0_181 or similar, check that pom.xml is not at the end of the POM field in the project configure Build (advanced) section.

- After it builds, you can click on the timestamp and select Console Output to see what happened.

 Configure
 Rename

 Recer

Permalinks

- [Last build /](#)
- [Last failed](#)
- [Last unsuc](#)
- [Last compl](#)

 **Build History** [trend](#) 

find x

#1

Jul 22, 2018 6:43 PM

 Changes

 Console Output

 Edit Build Information

 Delete Build

 Git Build Data

 No Tags

 Build Artifacts As Maven Repository

- Pages of this as it downloads and builds the project!

Downloaded from central: <https://repo.maven.apache.org/maven2/org/apache/ws/commons/axiom/axiom-parent/1.2.12/axiom-parent-1.2.12.pom> (18 kB at 52 kB/s)

Downloading from central: https://repo.maven.apache.org/maven2/org/apache/geronimo/specs/geronimo-activation_1.1_spec/1.0.2/geronimo-activation_1.1_spec-1.0.2.pom

Progress (1): 2.2 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/geronimo/specs/geronimo-activation_1.1_spec/1.0.2/geronimo-activation_1.1_spec-1.0.2.pom (2.2 kB at 6.4 kB/s)

Downloading from central: <https://repo.maven.apache.org/maven2/org/apache/geronimo/specs/specs/1.4/specs-1.4.pom>

Progress (1): 2.2/12 kB

Progress (1): 5.0/12 kB

Progress (1): 7.8/12 kB

Progress (1): 11/12 kB

Progress (1): 12 kB

Downloaded from central: <https://repo.maven.apache.org/maven2/org/apache/geronimo/specs/specs/1.4/specs-1.4.pom> (12 kB at 35 kB/s)

Downloading from central: <https://repo.maven.apache.org/maven2/org/apache/geronimo/genesis/config/project-config/1.2/project-config-1.2.pom>

Progress (1): 2.8/22 kB

Progress (1): 5.5/22 kB

Progress (1): 8.3/22 kB

Progress (1): 11/22 kB

Progress (1): 14/22 kB

Progress (1): 16/22 kB

Progress (1): 19/22 kB

Progress (1): 22 kB

Downloaded from central: <https://repo.maven.apache.org/maven2/org/apache/geronimo/genesis/config/project-config/1.2/project-config-1.2.pom> (22 kB at 62 kB/s)

Downloading from central: <https://repo.maven.apache.org/maven2/org/apache/geronimo/genesis/config/config/1.2/config-1.2.pom>



```
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent
[INFO] Compiling 222 source files to C:\Program Files (x86)\Jenkins\workspace\Test-Build\HighScoresServiceClient
[INFO] -----
[ERROR] COMPILATION ERROR :
[INFO] -----
[ERROR] No compiler is provided in this environment. Perhaps you are running on a JRE rather than a JDK:
[INFO] 1 error
[INFO] -----
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 01:48 min
[INFO] Finished at: 2018-07-23T17:39:44+10:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.1:compile (default-compile) on project HighScoresServiceClient: Compilation failure
```

```

sources\axis2\wsdl2code\src\com\itcuties\ws\HighScoreServiceStub.java file cannot be ove
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ HighScoresServiceC
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. bui
[INFO] Copying 1 resource
[INFO] Copying 800 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ HighScoresServiceClient
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is
[INFO] Compiling 222 source files to C:\Program Files (x86)\Jenkins\workspace\Test-Build
[WARNING] /C:/Program Files (x86)/Jenkins/workspace/Test-Build/HighScoresServiceClient/t
sources/axis2/wsdl2code/src/com/itcuties/serivces/impl/GetHighScoresResponseDocumentImpl
or unsafe operations.
[WARNING] /C:/Program Files (x86)/Jenkins/workspace/Test-Build/HighScoresServiceClient/t
sources/axis2/wsdl2code/src/com/itcuties/serivces/impl/GetHighScoresResponseDocumentImpl
for details.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 17.426 s
[INFO] Finished at: 2018-07-23T18:46:40+10:00
[INFO] -----
Finished: SUCCESS

```

If it can go wrong it will

- What this walkthrough shows is that DevOps tool chains and CI/CD is "bleeding edge" stuff.
- The field (and tool integration) is immature.
- Why?
 - Open source tools are not maintained by a single dev team
 - Updates/version support is not standardised (e.g. Jenkins only works with JDK8, but we're now up to JDK11!)
 - Single vendor solutions (Microsoft) are better but still not mature
- DevOps systems will break, and you need to be prepared to constantly re-educate yourself to maintain such systems.
 - Stack overflow, Dev forums, Git branches.

When the text book fails...

- Google/Internet search: AI overview, and so on ...

- That's all for this week.